Découvrez les bonnes pratiques de programmation avec les principes SOLID

Comprendre les pièges des solutions complexes

Voici un récit édifiant des programmes spatiaux américains et soviétiques. Réfléchissez à une solution à ce problème : comment écrivez-vous dans un environnement zéro-G ?

Les Américains et les Soviétiques avaient des réponses différentes.

- Des ingénieurs américains se sont mis au travail sur un stylo auto-pressurisé.
- Les Soviétiques ont opté pour un crayon.



L'excès d'ingénierie est un piège dans lequel vous pourriez tomber. 😅

Cet excès initial n'est pas le seul problème que vous pouvez rencontrer. Les systèmes logiciels deviennent de plus en plus complexes. Les utilisateurs veulent de nouvelles fonctionnalités et vous devez leur fournir. Mais l'ajout de ces fonctionnalités peut entraîner de mauvaises conceptions si vous ne faites pas attention.

Voici comment vous rencontrez des problèmes. La première fonctionnalité est facile à écrire. Vous déterminez ce qui doit être fait, le concevez et mettez le code en place.

Vient maintenant une autre fonctionnalité, qui ressemble beaucoup à la première, mais un peu différente. Comment intégrez-vous cette nouvelle fonctionnalité ? Eh bien, il y a toujours un copier/coller. Prenez le code qui existe, copiez-le et collez-le, puis corrigez-le un peu pour faire la nouvelle chose. Ensuite, répétez le processus pour les fonctions suivantes. A la longue, ce code devient difficile à maintenir.

Ou ajoutez la nouvelle fonctionnalité en ajoutant plus de complexité à une classe déjà existante. Puisque la classe fait déjà la majeure partie du travail, qu'est-ce qu'un peu de code en plus ?

Le problème est vraiment double.

- Premièrement, la solution gagne progressivement en complexité. Moins de gens peuvent comprendre comment cela fonctionne.
- Deuxièmement, la mentalité intrinsèque; l'équipe commence à accepter un travail moins qualitatif, dans le seul but de livrer une fonctionnalité. Donc, c'est plus qu'une mauvaise architecture ; c'est l'idée qu'une mauvaise architecture est OK et inévitable.

Comment éviter d'écrire des solutions qui vont trop loin ? Ou ceux qui deviennent difficiles à comprendre et à modifier ? 🤔

Rester simple. Une conception simple présente plusieurs avantages. Demandez simplement à n'importe quel mécanicien s'il préfère travailler sur une vieille super 5 turbo ou sur un moteur d'aujourd'hui. Un avantage est qu'il est plus facile à comprendre. Si c'est plus facile à

comprendre, alors c'est plus facile à modifier. De plus, vous pouvez être plus sûr que la modification ne cassera rien.

Un autre est qu'il est plus facile à tester. Si quelque chose est difficile à tester, cela signifie probablement que ce n'est pas simple. Une solution consisterait à diviser le test et le code suivant en éléments plus simples à tester.

C'est plus facile à dire qu'à faire. Heureusement, vous pouvez tirer parti des connaissances de ceux qui vous ont précédé et qui ont trouvé une meilleure façon de faire les choses. Les principes de conception SOLID capturent cette poignée d'idées.

Identifier les principes SOLID

Chaque lettre dans SOLID représente une excellente idée à garder à l'esprit lors de la conception de votre système. Nous examinerons chacun en profondeur au fur et à mesure de la progression du cours. Nous les mettrons également en pratique en créant une application de jeu de cartes simple. Passons en revue ces idées clés :

"S" est pour le principe de responsabilité unique
Une classe doit faire une chose, et la faire bien. Il ne devrait y avoir qu'une seule raison de la changer.

"O" est pour le principe ouvert/fermé
Une classe doit être ouverte à l'extension, mais fermée à la modification.

Qu'est-ce que cela signifie ?
Eh bien, chaque fois que vous ajoutez un nouveau concept au système (une extension), vous ne devriez pas avoir à revenir en arrière et à faire un tas de changements (modification) pour le prendre en charge.

"L" pour la substituabilité de Liskov
L'ajout d'un sous-type par héritage ne doit pas casser le code existant. C'est ce que j'appelle le principe "pas de surprises". Autrement dit, si le système fonctionne et que j'ajoute une nouvelle classe dérivée d'une autre, le système devrait toujours fonctionner.

"I" pour la ségrégation d'interface
Essentiellement le principe de responsabilité unique, appliqué aux interfaces.

"D" est pour l'inversion de dépendance
Les classes de haut niveau ne devraient pas avoir à changer lorsqu'une classe de bas niveau est modifiée. Les classes de haut niveau doivent définir une abstraction à laquelle la classe de niveau inférieur se conforme.

Nous allons implémenter chacun de ces principes en Java. Mais voyons d'abord comment ils sont intégrés dans une structure plus large : modèle-vue-contrôleur (MVC). Nous avons choisi MVC comme structure sur laquelle nous appuyer car elle s'aligne bien avec les principes SOLID.

Résumons!

- Des conceptions plus propres sont plus faciles à comprendre, à entretenir, à modifier et à tester.
- Suivre les principes SOLID conduit à une conception plus propre.
- Les principes SOLID sont :
  - Single responsibility
  - Open/closed principle
  - Liskov substitutability
  - Interface segregation
  - Dependency inversion

Structurer une application conforme à SOLID avec l'architecture MVC

Qu’est ce que MVC ?

MVC est une approche d'architecture logicielle. Il divise les responsabilités du système en trois parties distinctes :

- **Modèle** : Le modèle contient les informations d'état du système.
- **Vue** : La vue présente les informations du modèle à l'utilisateur.
- **Contrôleur** : le contrôleur s'assure que les commandes utilisateur sont exécutées correctement, en modifiant les objets de modèle appropriés et en mettant à jour les objets de vue.

Vous devriez déjà être en mesure de repérer l'un des principes SOLID à l'œuvre dans MVC : la responsabilité unique. 😎

Comment cela fonctionne-t-il en pratique ?
Regardons le fait d’aller au restaurant un instant. Tout d'abord, quelqu'un vous accueille à la porte et vous demande d'attendre qu'une table soit prête. Ensuite, ils vous emmènent à votre place. Une autre personne prend votre commande et finit par la livrer. Une autre personne le cuisine pendant qu'une autre nettoie après votre départ. Enfin, une autre personne lave tous les ustensiles de cuisine et la vaisselle.

Chaque personne a la responsabilité claire de vous fournir votre expérience culinaire. Vous n'avez pas besoin d'interagir avec toutes les personnes impliquées dans ce système. Le lave-vaisselle ne sort pas de l'arrière et vous aide à vous asseoir. Une seule personne n'est pas responsable de tous les travaux.

Sur la base de cet exemple, voici du Java qui violerait les principes SOLID :

```
1 class RestaurantWorker {
2    public void seatPatrons();
3    public void takeOrder();
4    public void cookOrder();
5    public void deliverOrder();
6    public void cleanTable();
7    public void washDishes();
8 };
```

Ce code a un seul objet (personne) dans le système qui fait tout le travail. Si vous ajoutez des emplois, je suis sûr que vous pouvez deviner où ils vont.

SOLID vous indique de répartir les responsabilités et le **modèle-vue-contrôleur (MVC)** indique où ces répartitions doivent être. C'est la base de **l'architecture** de votre système. Chaque classe du système appartient à l'une des trois catégories et a donc un **objectif spécifique.**

Que se passe-t-il dans le modèle (M) ?
Les informations d'état sont conservées dans des classes de **modèles**. Ce sont les éléments visualisés et manipulés. De plus, si vous avez besoin de stocker quoi que ce soit à long terme, ce seront les objets modèles. Si vous deviez programmer l'exemple du restaurant, quelques objets modèles seraient la commande d'un client ou le nombre de livres de pommes de terre dans le garde-manger :

```
class CustomerOrder {
    List<MenuItems> selectedItems;
    float baseCost;
    float tax;
};

class PantryItem {
    String name;
    float currentInventoryLevel;
    float reorderLevel;
};
```

Ou, si vous considérez un jeu de carte, l'état est toutes les informations qui racontent "l'histoire" actuelle. Combien y a-t-il de joueurs ? Quel joueur détient quelle carte ? Quelles cartes sont encore dans le jeu ? Toutes ces idées ajoutées ensemble constituent l'état du jeu.

Que se passe-t-il dans la vue (V) ?

La vue est la façon dont le modèle est présenté et interagit avec l'utilisateur. C'est la chose la plus susceptible de changer. Vous voulez une distinction claire dans la façon dont cette partie interagit avec le reste du système. Au restaurant, ce serait le menu, mais aussi l'accueil, les serveurs et toute autre personne avec qui vous interagissez directement. Si vous construisiez une application, ce serait votre interface utilisateur ou UI. Vous pourriez penser à l'une des vues comme suit : Tout ce qui précède reflète les interactions que l'utilisateur verrait.

```
class Greeter {
    public void askHowManyPeopleToSeat();
    public void reportEstimatedWaitTime();
    public void takeCustomersToTable();
}
```

Que se passe-t-il dans le contrôleur (C) ?

Le contrôleur est l'endroit où le flux de l'application est géré. Tout l'enchaînement des interactions entre l'utilisateur et le système est là. L'utilisateur interagit avec la vue, qui interagit ensuite avec le contrôleur. Le contrôleur apporte ensuite les modifications appropriées aux objets du modèle, en crée de nouveaux ou supprime ceux qui ne sont plus nécessaires. Dans l'exemple du restaurant, le contrôleur serait l'ensemble de règles, établies par le propriétaire ou le gérant, sur la façon de prendre soin d'un client :

```
class ProcessCustomer {
    public void customerGreeted();
    public void customerSeated();
    public void orderTaken();
    public void orderDelivered();
    public void patronSatisfactionChecked();
    public void billPresented();
    public void billPaid();
};
```

Cela représenterait le flux de travail ou la séquence d'étapes à travers laquelle le client serait guidé.

Quels sont les avantages du MVC?

Si vous suivez ce modèle MVC, vous construisez un système avec des responsabilités claires et utiles. L'un des avantages de l'utilisation de MVC est qu'il suit le principe S dans SOLID. Que signifie encore le S? Responsabilité unique. Mais la grande idée est que les classes qui sont des modèles font une chose, tandis que celles qui sont des vues ou des contrôleurs en font d'autres.

Mais il suit également les autres principes. Par exemple, puisque la vue est séparée, il est facile d'ajouter de nouveaux éléments d'interface utilisateur sans avoir à modifier le modèle ou le contrôleur.
De quel principe s'agit-il? Le principe ouvert/ferméEt les autres principes sont également utilisés.

Récapitulons!

MVC suit le « S » des principes SOLID en séparant les responsabilités. Le modèle contient des informations d'état. La vue contient les éléments qui interagissent avec l'utilisateur. Le contrôleur s'assure que la séquence d'étapes se déroule correctement.